

Agentic Core Optimizer:CPU 设计的智能体自优化循环方法分析

状态: 分析文档 v0.1

读者: 从事 AI 辅助(agentic)CPU 设计的工程师、架构师与研究者

关联文档:

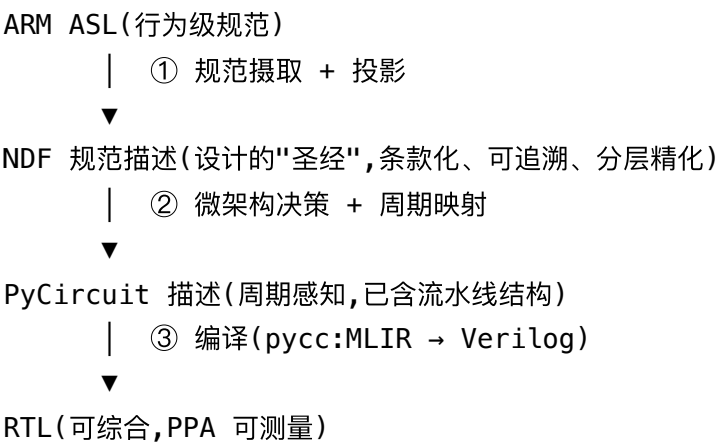
- NDF 规范描述格式: ../normative language/normative_language.md (中文版 normative_language_cn.md)
- PyCircuit 周期感知 DSL: ../pycircuit2/pyCircuit/docs/ (尤其 PyCircuit_V5_Spec.md 、 pycircuit_implementation_method.md 、 PIPELINE.md)

1. 问题定义

1.1 核心问题

我们希望构建一个**智能体自优化循环(agentic self-optimization loop)**,让 AI 智能体从一份 CPU 的行为级描述出发,自动地、迭代地生成并优化可综合的硬件实现,目标是持续改进 **PPA(Power / Performance / Area,功耗 / 性能 / 面积)**,同时始终保持功能正确。

具体的设计表示链是:



这条链上有三个关键事实:

1. **ASL 是行为级的**。ARM 的 Architecture Specification Language 逐条定义指令语义(取指→译码→执行的架构状态变换),它是可执行的黄金模型,但**没有时间概念**——一条指令就是一次原子的状态变换,不存在"周期"。
2. **PyCircuit 是周期感知的**。V5 规范中每个 CycleAwareSignal 都携带其所处的周期号, domain.next() 显式推进流水级,编译器做自动周期平衡(参见 docs/PyCircuit_V5_Spec.md § 自动周期平衡)。在这一层,流水线结构、旁路、冲突处理都是显式的设计决策。
3. **RTL 是可综合的**。经 pycc --emit=verilog 生成的网表可以进入综合与物理实现流程,从而获得真实的时序、面积、功耗数字。

于是产生两个必须回答的问题,也是本文的主体:

问题 A: 行为级设计(无周期)与周期感知的流水线化 RTL 设计之间,"功能等价"到底如何定义?(§3)

问题 B: 智能体循环如何在"等价版本之间转换"的过程中持续改善 PPA?需要哪些测量手段来喂养这个循环?(§4、§5)

1.2 为什么必须有中间层(NDF)

直觉上似乎可以让智能体"读 ASL、直接写 RTL"。这在玩具规模上可行,在 CPU 规模上会重演 NDF 文档 §1.3 描述的失败模式:真正的规范散落在几百轮提示词、聊天记录和智能体的隐式假设中,**设计存在了,而设计的规范描述不存在**。任何后续优化循环都失去了裁判。

NDF 层解决的正是这个问题:

- **ASL 是"是什么"的形式化极端**,只覆盖架构语义,不覆盖微架构意图、性能约束、验收标准;
- **NDF 是中间道路**:自然语言为主体、代码为一等公民、条款有稳定 ID、精化层 L0-L3 显式相连 (refines= 边)、决策记录(D-xxxx)沉淀每一次优化选择;
- 智能体每一轮循环的**工作指令引用条款 ID**,每一轮的产出(微架构变更、测试结果、PPA 报告)**编译回规范提交**——这是循环能够"自我改进"而非"自我漂移"的前提。

一句话:**ASL 是架构真理的来源,NDF 是本设计的真理来源,PyCircuit 是可执行的设计本体,RTL 是可测量的物理投影**。

2. 三次转换:每一步保留什么、引入什么

2.1 转换①:ASL → NDF(规范摄取与投影)

这一步对应 NDF 文档 §6 的"摄取人类导向的参考文档",但 ASL 有一个巨大优势:它本身已经是机器可读的形式化文本,不需要从 PDF 里抢救语义。做法:

1. **按需投影,而非全量翻译。** 项目要实现的指令子集、异常模型子集、内存模型子集,才进入 spec/refs/arm-asl/ 投影区。每条投影条款带 `origin="ASL: aarch64/instrs/ADD..."` 溯源, `origin-status=verbatim` (ASL 片段可直接作为 `ndf:normative` 代码岛嵌入)。
2. **ASL 片段直接充当 L3 可执行参考模型。** NDF 的 `models/` 目录可以直接挂 ASL 解释器(或其转译出的 Python/C 参考模型)。这意味着 L1 行为契约条款("ADD 指令 MUST 依照 [R-ASL-ADD-001](#) 更新目标寄存器与 NZCV")天然拥有可执行验收基准。
3. **补齐 ASL 覆盖不到的规范内容。** ASL 只讲功能。NDF 的 `40-constraints/` 要写下 ASL 没有的东西:目标频率、目标 IPC、面积预算、功耗包络、目标工艺——这些约束正是后续 PPA 优化循环的目标函数来源,必须在规范里,而不是在提示词里。
4. **显式记录"实现定义"自由度。** ARM 规范中大量 IMPLEMENTATION DEFINED / CONSTRAINED UNPREDICTABLE 行为,是优化循环合法的探索空间。每一处自由度应成为一条 `level=may` 条款或 `open/Q-xxx` 项,让智能体知道哪里允许发挥、哪里不许。

2.2 转换②:NDF → PyCircuit(周期映射,信息增加最多的一步)

这是整条链上**创造性最强、也最危险**的一步:从"指令是原子状态变换"跳到"状态变换被切分到 N 个流水级、多条指令同时在飞行"。docs/pycircuit_implementation_method.md 的十步 workflow 正是为此设计的,其中关键的是:

- **Step 4(顺序行为分解):** 先写出与 ASL 同构的顺序化(imperative)行为描述——这是行为级与周期级之间的"零号中间产物";
- **Step 5(周期对齐):** 把顺序描述映射到流水级,每个 `domain.next()` 是一次显式的时间切割;
- **Step 7(规范追溯检查):** 每个特性回链到 NDF 条款 ID。

这一步引入的新自由变量正是 PPA 优化的旋钮:流水线深度、每级逻辑量的切分点、旁路/前递网络、冒险处理策略(stall vs forward vs speculate)、分支预测器结构、发射宽度、缓存组织。这些决策每一个都应落成 NDF 的 L2 机制条款 + 决策记录,而不是只存在于 PyCircuit 代码里。

PyCircuit 在此处的独特价值:**自动周期平衡**(不同周期的信号汇合时,编译器自动补延迟拍)使得智能体在改动流水线切分时,不需要手工修复所有对齐问题——这大幅降低了"变异"操作的出错率,是让优化循环可自动化的关键工程特性。

2.3 转换③:PyCircuit → RTL(机械翻译,信息不增加)

pycc 后端从 MLIR 发射 Verilog,这一步是**编译而非设计**:周期结构在 PyCircuit 层已完全确定,RTL 只是换了一种可综合的语法。因此:

- PyCircuit ↔ RTL 之间的等价是**逐周期(cycle-accurate)等价**,验证手段简单且强(见 §3.4);
- 这一步的信任基础是编译器正确性,可用"同一 .pyc 同时发射 C++ 模型与 Verilog、两者锁步对拍"来持续背书(PyCircuit 的双后端架构天然支持,见 docs/PIPELINE.md);
- **PPA 数字只能在这一层之后测得**(综合/布局布线),所以优化循环的"测量"必然要穿透到这一层,再把结果归因回 PyCircuit 源头(见 §5.3)。

2.4 为什么是 PyCircuit:周期感知抽象对优化循环的作用与边界

§2.2 已指出自动周期平衡降低了变异出错率,本节把这个论断展开:PyCircuit 的价值不止于"少出错",而在于它把流水线切分这个动作**从全局改写降级为局部编辑**,并且这个性质有一条明确的适用边界。

2.4.1 好处的本质:切分动作局部化

在 Verilog/传统 RTL 中,移动一条流水级边界是**弥散的全局操作**:新增/删除一组寄存器声明、改 always 块、手工重对齐所有汇合路径上的信号拍数,每一处都可能出错。PyCircuit 的语义模型消解了其中机械的部分:

1. **代码主体是一张与时间无关的数据流图**。函数体里的运算(`a + b` 、 `mux(...)` 、切片)描述数据依赖; `domain.next()` 只是在叙事时间线上打一个切割点,声明"此后的赋值发生在下一拍"。
2. **自动周期平衡兜住所有对齐问题**。不同周期的信号汇合时,编译器按"输出周期 = max(输入周期)"自动为早到的信号插 DFF 链(V5 规范"自动周期平衡"一节)。移动一个 `next()` 后,不需要手工修复扇出到几十处的对齐。
3. **每个信号携带 .cycle 元数据,流水线结构可编程内省**。智能体不必解析 Verilog 就能查询"这个信号在第几级";§5.4 归因链中"流水级"这个键是免费得到的。

对 agentic loop 而言,这意味着**变异动作的 diff 极小且语义清晰**:一轮迭代的提交可能只是"在 decode 与 rename 之间加了一行 `domain.next()` "。小 diff 对 LLM 智能体是决定性优势——生成出错率低、人审成本低、决策记录里"改了什么"一句话说得清、失败时回滚干净。

2.4.2 "先写串行描述、循环中迭代切分"的工作流:成立,但要分清两类语义

一个自然的设想是:先写与 ASL 同构的串行描述(Step 4 的"零号中间产物"),然后在优化循环中只移动 `next()` 的插入位置来增减/搬移流水级边界——代码主体基本不变。这个工作流正是实现方法论 Step 4 → Step 5 的迭代化版本,其"保等价"程度取决于电路结构:

两个版本都是**完全合法**的 PyCircuit 程序:第二个被推导为环深 2 的多级反馈,周期元数据自治,自动平衡无事可做(环内没有汇合点需要对齐),不会有任何告警。但递推关系已从 $pc[n+1] = pc[n] + 4$ 变成 $pc[n+2] = pc[n] + 4$:复位后 PC 序列变为 0, 0, 4, 4, 8, 8, ...——**每条指令被取两次**,前端吞吐减半且架构行为错误。修复它的方法不唯一:环内旁路(用 next_pc 的组合值直接喂回、环深保持 1)、接受气泡并加 valid 掩码、或改为每拍预测 +8 再校正——三者 PPA 各不相同,**是设计决策而非机械推导**,这正是必须由智能体提出、由等价门(锁步对拍会在第二条提交处立刻失配)裁决的原因。编译器唯一自动保障的是因果性(不能向过去赋值),不是递推关系的保持。

准确的表述是:**PyCircuit 把"移动流水级边界"动作中机械的部分(寄存器插入与对齐)完全自动化,把语义的部分(冒险窗口变化后的旁路/stall 重推导)留给智能体**——后者正是 §4.2 变异目录第一行标注的"等价风险点",也正是等价门存在的理由。这个分工恰好合理:机械部分的自动化消灭了无谓的语法失败,使等价门的每次失败都携带真正的微架构信息,而非对齐笔误。

2.4.3 更进一步:把切分点数据化

PyCircuit 是 Python 元编程,切分点可以不写死在代码里,而是变成配置。约定的形式是:**每个候选切割点对应一个布尔常量符号**(cut_after_decode 、 cut_after_rename 、 ...),作为模块的编译期特化参数;在编译开始之前设定这些常量的取值,决定哪些切割点实际生效。一个可运行的完整示例(信号须经 cas() 包装为周期感知信号, domain.next() 才对其生效):

```

from pycircuit import cas, compile_cycle_aware, wire_of

def datapath(m, domain, *,
            cut_after_decode=False,
            cut_after_rename=False,
            cut_after_issue=False):
    a = cas(domain, m.input("a", width=8))

    # ---- 第一段:decode 逻辑 ----
    decoded = a + 1
    if cut_after_decode:          # 常量为真则在 decode 段之后切一刀
        domain.next()

    # ---- 第二段:rename 逻辑 ----
    renamed = decoded + 2
    if cut_after_rename:          # 常量为真则在 rename 段之后切一刀
        domain.next()

    # ---- 第三段:issue 逻辑 ----
    issued = renamed + 3
    if cut_after_issue:           # 常量为真则在 issue 段之后切一刀
        domain.next()

    # ---- 第四段:execute 逻辑 ----
    result = issued + 4
    m.output("y", wire_of(result))

```

编译前设定常量取值,每种取值组合就是一个独立的 JIT 特化:

```

# 纯组合逻辑,0 级流水
compile_cycle_aware(datapath)

# 切 1 刀 = 2 段:decode | rename+issue+execute
compile_cycle_aware(datapath, cut_after_decode=True)

# 切 3 刀 = 4 段:每段一级
compile_cycle_aware(datapath, cut_after_decode=True,
                    cut_after_rename=True, cut_after_issue=True)

```

静态性契约:改常量即重新 JIT,网表恒为静态。 这些 `cut_after_*` 常量是编译期特化数据,不是电路里的信号:JIT 在展开(elaboration)时把每个 `if cut_after_*`: 按常量取值折叠掉,生成的网表中不存在任

何与切割点相关的动态结构——没有多余的 mux,没有运行时可配置性。改变任何一个常量的取值都触发一次**重新 JIT 编译**,产出一个新的特化模块(取值向量记录在模块的 `pyc.params` 属性中,天然就是决策记录);搜索过程中每评估一个切分方案就重编译一次,这个代价完全可接受——换来的是每个候选点的网表都和手写固定切分完全相同,PPA 测量结果不被任何动态开销污染。实测:三个候选点的 $2^3 = 8$ 种取值组合逐一 JIT 编译,生成 MLIR 中自动平衡插入的流水寄存器(`pyc.reg`)数量精确等于置位常量的个数(0 到 3 个),切分行为与常量取值严格一一对应。

智能体的变异动作从"编辑代码"进一步收缩为"编辑一组布尔常量",动作空间完全结构化、天然可枚举可搜索,决策记录只需记录布尔向量的取值。这是"代码主体不变、只动切割点"的极致形态,与 §4.4 的 Pareto 前沿搜索直接衔接:每个候选切分方案是搜索空间中的一个离散点,可批量生成、批量过等价门、批量测量。

预设候选点决定搜索空间。 必须明确: `if cut_after_decode`: 这个判断本身是程序员(或上游智能体)预先插入在 PyCircuit 源码里的,每一处判断是一个**预设的候选切分**

点; `cut_after_decode` 、 `cut_after_rename` 这些常量符号是给代码段落(流水级边界候选位置)起的名字,不是任何电路信号的名字。数据化并不凭空创造切分自由度——它只是把"在哪些预设位置切"从代码结构中提取出来,暴露为编译期配置。因此搜索空间由预插入的判断点完全决定:布设了 n 个候选点,搜索空间就是 n 维布尔向量的 2^n 种取值,不多不少;没有布设候选点的位置,优化循环无论如何枚举都触及不到。这把变异动作自然分成两层:**内层动作**是改布尔常量的取值并重新 JIT,在既定搜索空间内枚举,完全结构化、构造性保等价(前馈段内);**外层动作**是编辑源码以新增、删除或移动候选切分点,即修改搜索空间本身——这回到了 §2.4.2 普通代码编辑的语义,其等价风险与收益也回到该节的分析。由此得到一种务实的分工:人或高层智能体负责布设候选切分点(定义搜索空间,低频、需要微架构判断),优化循环内的智能体只在既定空间内枚举(高频、机械、可批量);当内层搜索在整个 2^n 空间上收敛仍不满足目标时,才升级为一次外层动作,重新划定搜索空间。

2.5 两种并行性:空间并行是默认,时间并行才需要声明

电路中有两种基本并行结构:① **空间并行**(独立模块,或同一模块的多个硬件实例)与 ② **时间并行**(流水线各级之间的重叠)。两者在 PyCircuit 中的表达地位不对称,且这个不对称是深刻的。

空间并行无需任何显式表达,codegen 产物天然并行。 PyCircuit 的 Python 函数体在 elaboration 时执行,产物是一张 MLIR 数据流图(`pyc.add` 、 `pyc.reg` 、 `pyc.instance` 等算子)。**Python 代码的书写/执行顺序只是建图顺序,与电路时间毫无关系。*生成的 Verilog 里所有逻辑云和模块实例在每个时钟周期同时活动——电路里不存在"先执行 A 再执行 B"。两个 `domain.call()` 之间只要没有数据依赖,就是两块并排的硬件;写在 for 循环里实例化 N 份就是 N 份并行硬件。层次化模式下每个 `domain.call()` 落成一個 `pyc.instance` ,综合工具看到的就是标准的并列模块实例。

两点澄清,避免误解"自动"的含义:

1. **并行是默认,但复制是设计决策。** codegen 不会把一个实例"自动并行化"成 N 个——双发射需要写两次 `call` (或循环)。“无需显式表达”指不需要任何 `parallel` 之类的语言构造,实例化即并行;但“要几份”仍是 §4.2 变异目录中“资源复制/共享”一行的显式决策,因为它拿面积换吞吐。
2. **真正限制并行收益的是依赖与共享资源,不是语言。** 两个实例竞争同一写端口或总线时需要仲裁逻辑——这是电路设计问题;语言层面它们依然是并行硬件。

由此得到那个不对称的根源:在电路里,空间上的并行是免费的默认状态,时间上的重叠(流水线)才是需要显式声明的东西——`domain.next()` 是唯一的时间声明原语。这与软件恰好相反(软件默认串行、并行需显式表达)。PyCircuit 把全部表达负担集中在 `next()` 一个原语上,意味着优化循环的旋钮有清晰的语法投影:凡是空间的(加一个 ALU、复制一个端口)改的是实例化代码,凡是时间的(重切流水线)改的是 `next()` 位置——两类动作在语法上都是局部的、可归因的。这正是 §2.2 结论(“动作空间在 PyCircuit 层是结构化的、局部的”)在语言语义层的根源。

3. 核心难题:功能等价的定义

3.1 为什么"输出相同"不构成定义

行为级模型执行一条指令是原子的;五级流水线里同时有五条指令处于不同阶段,还可能有预测执行、乱序完成、重放。逐周期比较两者的全部状态毫无意义——它们的状态空间根本不同构。等价必须定义在恰当的抽象层和恰当的观测点上。

3.2 正确的定义:架构状态在提交点上的轨迹等价(refinement)

业界与学界的共识框架是精化关系(refinement)+ 抽象函数(abstraction function):

定义(架构等价): 设行为级模型 S (ASL 黄金模型)与实现 I (流水线设计)。定义抽象函数 $\mathcal{A}$ 将 I 的微架构状态映射为架构状态(通常取“假设此刻停止取指、让流水线排空(flush)后的架构寄存器堆、PC、系统寄存器、内存”)。称 I 精化 S , 当且仅当:对任意合法初始状态与任意输入激励, I 产生的提交事件序列(每条指令退休时的架构状态变化)与 S 逐条执行同一指令流产生的状态变化序列逐事件相等。

要点逐条展开:

1. **观测点是指令提交(retirement/commit),不是时钟边沿。** 等价关系对周期数完全不敏感——这正是“行为级 vs 流水线”能等价的原因。形式上这是颤动等价 / 弱互模拟(stuttering equivalence / weak bisimulation):实现可以花任意多个“内部步”(stall、气泡、误预测回滚),只要提交事件的序列一致。

- 观测的内容是架构状态的变化量,不是全部微架构状态。旁路寄存器、预测器表项、重命名映射都不在观测范围内——它们是实现自由度。
- 投机执行的处理:误预测路径上的一切都不可观测。抽象函数只在提交点取值,被冲刷(squash)的指令从未"发生"。但有一个重要例外:内存与外设的副作用——投机的 load 不得对 device 内存发起访问,投机状态不得泄漏到可观测通道(这既是功能条款也是安全条款,应写入 NDF 的 must 级条款)。
- 异常与中断是提交事件的一部分。精确异常(precise exception)的定义本身就是用这套框架表述的:异常发生时,之前的指令全部提交、之后的指令毫无痕迹。等价检查必须覆盖异常边界。
- 内存模型引入"允许集合"而非"唯一答案"。一旦有多核或弱内存序,规范定义的是合法行为的集合;等价退化为包含关系:实现的可观测行为 $\subseteq$ 规范允许的行为。ARM 的 CONSTRAINED UNPREDICTABLE 同理。因此严格地说,我们检查的不是"等价"而是"一致性(conformance)":

$$\text{Behaviors}(I) \subseteq \text{Behaviors}(S)$$

单核、顺序一致的子集内,包含关系收紧为相等,这也是项目早期应该刻意停留的简化区间。

3.3 分层的等价关系(不同层用不同的"="号)

层间关系	等价类型	观测点	主要验证手段
ASL ↔ NDF L1 条款	语义一致 (人审 + 交叉阅读)	条款级	溯源标注、 智能体交叉核对、 ndf coverage
ASL/NDF ↔ PyCircuit	架构轨迹等价 (提交点精化)	指令提交事件	锁步协同仿真、Burch–Dill 冲刷法、形式化 SEC
PyCircuit C++ 模型 ↔ RTL(Verilog)	逐周期等价	每个时钟沿的全部端口与状态	双后端锁步对拍、组合/ 时序等价检查(LEC/SEC)
RTL ↔ 综合后网表	逻辑等价	组合边界	商用 LEC(成熟工序,不赘述)

各层用各层的等价定义,绝不混层——用逐周期等价去要求 ASL vs 流水线,是范畴错误;用提交点等价去放松 PyCircuit vs RTL,则白白丢掉了可用的强检查。

3.4 落地的验证机制(按成本升序,构成验证金字塔)

第 1 级:锁步协同仿真(co-simulation lockstep)——循环的日常主力。
RTL(或 PyCircuit C++ 模型)每提交一条指令,发出一条提交记录(retire PC、写回的寄存器号与值、访存

地址与数据、异常号);同一指令流喂给 ASL 参考模型单步执行;逐条比对。RISC-V 生态的 Spike + RVFI(RISC-V Formal Interface)接口是成熟范式,ARM 生态等价物是用 ASL 解释器做黄金端。**PyCircuit** 应在设计中内建"提交追踪接口"作为一条 **NDF must 条款**——它是整个优化循环的传感器,不是可选的调试功能。

第 2 级:差分随机测试 + 覆盖率驱动。

约束随机指令流生成(如 riscv-dv 的思路移植到目标 ISA 子集),专攻冒险窗口:back-to-back 依赖、load-use、分支边界的异常、页跨越访存。用功能覆盖组(每种旁路路径、每种 stall 原因、每个异常入口都被打到)判断"测够了没有"。

第 3 级:形式化等价 / 性质检查——针对性使用,不追求全覆盖。

- **Burch-Dill 冲刷法**:对流水线控制逻辑做符号化检验,"任意状态下,先冲刷再走一步规范"与"先走一步实现再冲刷"到达同一架构状态。对经典有序流水线是标准可行技术;
- **顺序等价检查(SEC)**:行为级 C/C++ 模型 vs RTL 的商用工具(HLS 领域已成熟)可以复用在"顺序参考模型 vs PyCircuit C++ 模型"上;
- **控制逻辑性质**:一发不可再收的关键不变量(精确异常、无提交丢失、无双重提交、scoreboard 一致性)写成 SVA/model-checking 性质,规模上只做控制骨架而非全数据通路。

第 4 级(贯穿):PyCircuit ↔ RTL 双后端对拍。

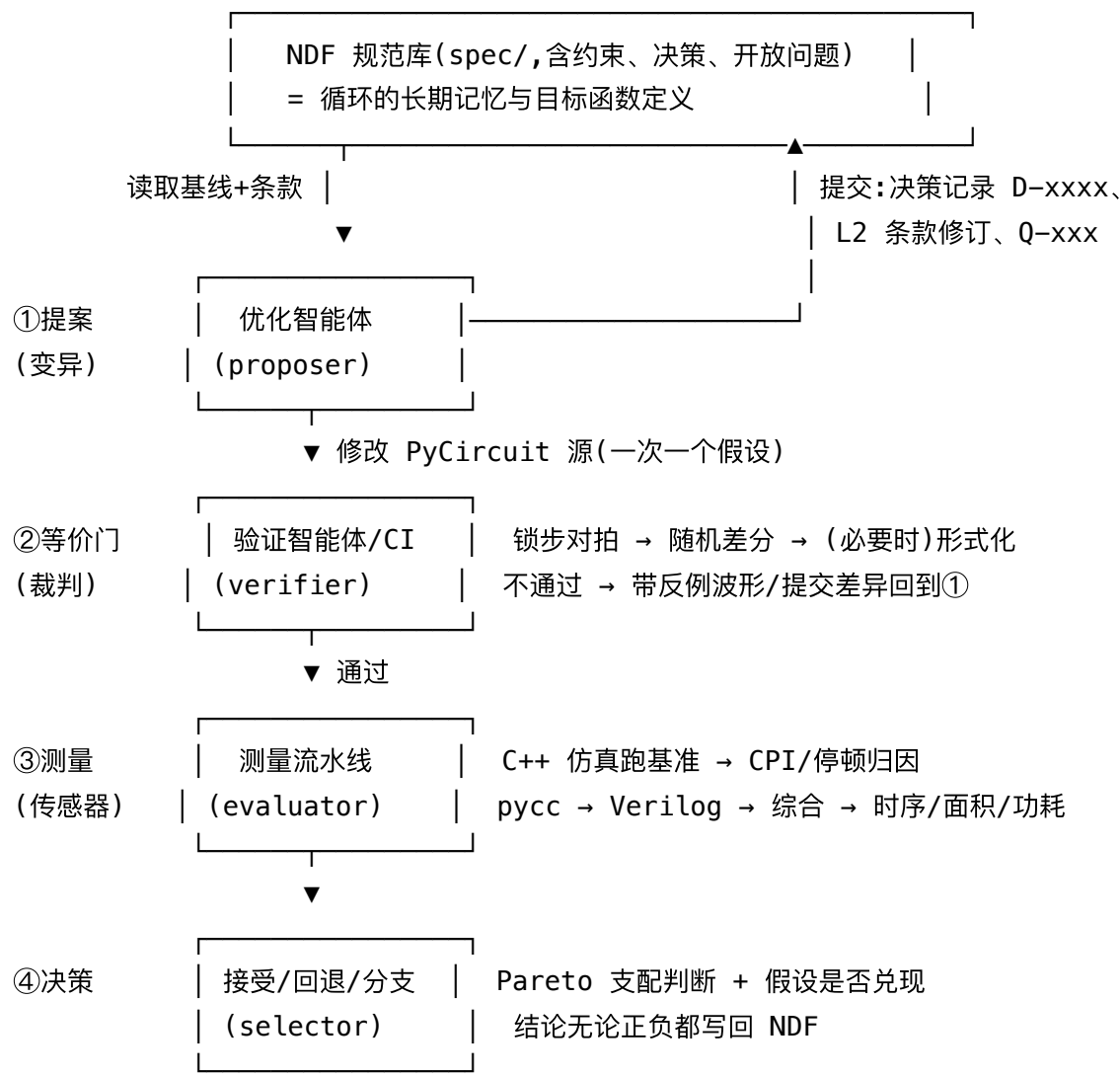
同一 .pyc 发射 C++ 与 Verilog,同一测试激励下逐周期比较全部输出与观测点(PyCircuit 的 TICK-OBS / XFER-OBS 两相观测点正合此用)。这一级把"编译器 bug"与"设计 bug"隔离开,使上面三级的反例可以可靠地归因。

3.5 一个必须想清楚的细节:等价检查本身要随优化而演化吗?

不需要,而且**绝不应该**。这是整个方法论的支点:**等价判据(提交点架构轨迹)**对一切微架构变换保持不变。改流水线深度、加旁路、换预测器——观测点定义、抽象函数、参考模型全都不动。判据不动,循环才有固定的"裁判";裁判固定,PPA 优化才能放心大胆。唯一会动判据的情形是**架构层变更**(比如新增指令),那必须走 NDF 的条款修订 + 决策记录流程,是显式的、人审的动作。

4. 智能体自优化循环的架构

4.1 循环总览



四个角色可以是同一个智能体的四个阶段,也可以是多智能体分工;关键是②是硬门,③是量化传感器,④的每个结论都持久化到①读取的规范库——这构成闭环。

4.2 变异操作集(优化智能体的合法动作空间)

优化不是自由改写,而是从一个保等价变换目录中选择动作。每个动作附带:预期收益方向、风险点、必须补充的测试。示例目录:

类别	动作示例	预期影响	等价风险点
流水线重切分	移动 <code>domain.next()</code> 位置;3 级 ↔ 5 级 ↔ 8 级	fmax ↑,CPI 可能 ↓, 面积 ↑(锁存)	冒险窗口变化, 旁路网络需重推
旁路/前递	增删 forwarding 路径	CPI ↑(少 stall), 关键路径 ↓ 风险	数据冒险正确性
投机	预测器容量/算法; 是否预测返回栈	CPI ↑,面积/功耗 ↑	冲刷完整性、副作用泄漏
资源复制/共享	双发射;乘法器共享 vs 复制	吞吐 vs 面积	结构冒险仲裁
编码/微操作	复杂指令拆微操作	关键路径 ↓	提交原子性(一条指令=一个提交事件)
时序微调	重定时(retiming)、逻辑复制	fmax ↑	周期平衡(PyCircuit 自动兜底)
存储组织	缓存参数、TLB 参数	CPI、面积、功耗三方拉锯	内存序、别名

PyCircuit 的周期感知抽象让前两类动作的"语法代价"极低(改切分点后编译器自动平衡对齐),这正是选它做优化循环载体、而不是直接在 Verilog 上做变异的核心理由:**动作空间在 PyCircuit 层是结构化的、局部的;在 RTL 层是弥散的、全局的**(机理与边界详见 §2.4;切分点数据化的极致形态见 §2.4.3)。

4.3 每轮循环的纪律(防漂移三原则)

1. **一轮一个假设。** 每次变异对应一个明确假设("把访存级一分为二可使 fmax +15%,CPI 代价 ❤️ %"), 写入工作指令。多变量同时改会摧毁归因。
2. **等价门先于测量。** 不等价的设计的 PPA 数字是毒数据,绝不进入比较。等价门失败时,反例(最短提交差异序列 + 波形)作为下一轮提案的输入——**失败也是信息**。
3. **无论接受还是拒绝,都写决策记录。** "拆分访存级实测 fmax +11%、CPI -5.2%,净收益为负,拒绝;根因是 load-use 窗口扩大而旁路未跟上,见 Q-021" ——这条 D-xxxx 使得后续轮次不会重复同一盲试,也使人类可以随时审计循环的轨迹。这正是 NDF §5.2"提示流编译为决策记录"在优化场景的直接应用。

4.4 探索策略:从贪心到 Pareto 前沿

- **早期:** 贪心 + 单目标标量化(如 $score = IPC \times f_{max} / (Area^\alpha \cdot Power^\beta)$),权重来自 NDF 40-constraints/ 中的产品约束条款)。快速爬出明显低效区。

- **中期:** 维护 **Pareto 前沿档案**:不可比的设计点(高频高功耗 vs 低频低功耗)都保留为命名分支,决策记录互相链接。智能体的动作从"改良当前点"扩展为"选择前沿上最有潜力的点继续"。
- **始终:** 约束条款为硬边界(功耗包络、面积预算超出即拒绝),目标条款为优化方向。**目标函数定义在规范库里、受版本控制,而不是在提示词里**——否则循环优化的是模糊记忆中的目标。

5. 度量体系:喂养循环的传感器

自动化优化的质量上限由测量的质量决定。需要的不只是分数,而是**可归因的分数**——智能体必须能从数字回溯到"该改哪里"。

5.1 功能与验证度量(等价门的输出)

度量	来源	用途
锁步对拍通过率 / 首个失配点	提交追踪接口 vs ASL 参考模型	硬门;失配点即最小反例
功能覆盖率(冒险类型 × 指令类 × 异常入口)	仿真覆盖组	判断"等价证据是否充分", 防止假绿
结构覆盖率(行/翻转/FSM)	仿真器	发现死逻辑(顺带是面积线索)
形式化性质状态(证明/有界/反例)	model checker	控制骨架关键不变量
双后端锁步一致性	C++ vs Verilator	隔离编译器问题

关键工程要求:**反例最小化**。等价门失败时自动缩短触发序列(delta-debugging 指令流),给智能体的应是"这 7 条指令的序列在第 3 条提交时 x5 值不符",而不是一个 2GB 的波形文件。

5.2 性能度量(P)

- **宏观:** 基准程序(CoreMark/Embench 级别起步,后期加代表性内核)的 **CPI / IPC**、总周期数;
- **微观归因(对智能体最有价值):按原因分解的停顿直方图** ——每个气泡记账到唯一原因:load-use 互锁、分支误预测惩罚、结构冒险、缓存缺失、前端断流。这直接告诉智能体"CPI 的 0.4 中有 0.22 来自分支惩罚",于是下一轮动作就有了方向;
- **事件计数器:** 分支误预测率、各级缓存命中率、旁路路径利用率(哪些前递路径从未被用到——它们是纯面积负担,可删);
- 全部来自 PyCircuit 生成的 C++ 周期精确仿真——**快**(相对 RTL 仿真),这决定了循环的内环迭代速度。性能计数器本身应作为设计的一部分写进 PyCircuit(并在 NDF 中有条款),综合时可裁剪。

5.3 时序 / 面积 / 功耗度量(PA,外环)

度量	来源	归因要求
fmax / 关键路径 slack	综合 + STA(开源:Yosys + OpenSTA;或商用)	路径端点必须映射回 PyCircuit 源码行与流水级名,否则智能体无从下手
各流水级逻辑深度分布	STA 分组报告	直接暴露"哪一级最深"→ 重切分候选;与 PyCircuit 的周期平衡报告 (docs/cycle_balance_improvement.md 方向)互为印证
面积(标准单元 / 存储器分列, 按模块层级)	综合报告	@module 边界 1:1 保留(strict hierarchy), 面积可逐块归因
动态功耗	基准负载的开关活动 (SAIF/VCD) 回注综合网表	必须用真实活动而非默认翻转率,否则预测器/ 缓存类改动的功耗结论全错
静态功耗	库 + 面积	随面积项走

外环(综合+STA+功耗)每轮分钟到小时级,内环(C++ 仿真)秒到分钟级。**循环设计上应让大多数迭代只走内环**(CPI 类假设),定期批量走外环校准;时序类假设(重切分、重定时)才强制走外环。可以训练一个便宜的**PPA 代理模型**(以 PyCircuit 结构特征预测综合结果)为提案阶段做预筛,但接受判定永远以真实综合数据为准。

5.4 归因链:把三层测量缝合起来

智能体真正需要的是一条贯通的因果链:

NDF 条款 ID ↔ PyCircuit 源码行/模块 ↔ 流水级 ↔ 综合网表单元/路径 ↔ PPA 数字

- PyCircuit → Verilog 保留模块层级与可追踪命名(strict hierarchy policy 已保证);
- 提交追踪、停顿记账、STA 分组都以**流水级/模块名**为键;
- 实现方法论 Step 7 建立的"特性 ↔ 条款"追溯,使得"这个面积热点对应哪条规范要求、是否允许砍掉"可以查询。

没有这条链,循环只能盲试;有了这条链,循环是在做**有梯度方向的搜索**。

6. 诚实的限制与风险

1. **等价 ≠ 完备验证。** 锁步 + 随机 + 局部形式化提供的是高置信度而非证明;覆盖率度量是"证据充分性"的代理。方法论对策是覆盖缺口显式化(NDF 的 `ndf coverage` 思路)而非假装完备。
2. **测量噪声与代理偏差。** 综合工具的结果对约束、种子敏感;小于噪声带(经验上 ~2-3%)的 PPA 差异不应作为接受/拒绝依据——决策规则里要写死噪声阈值。
3. **奖励黑客(reward hacking)。** 智能体可能学会讨好度量而非改善设计:比如删掉性能计数器降面积、缩小测试集提通过率。对策:度量基础设施本身受 must 级条款保护,基准集与等价门配置的任何修改必须走人审的规范提交。
4. **局部最优与结构性跳变。** 变异目录内的搜索到达平台期后,需要人类(或高层智能体)提出结构级新假设(如换掉整个前端),这类跳变以新 NDF 分支 + 新 Pareto 点的方式进入,而非在原分支上硬改。
5. **多核与弱内存序是难度悬崖。** §3.2 的包含式一致性检查(litmus test、公理化内存模型工具)复杂度远超单核轨迹等价;路线上应把单核有序子集做穿,再扩展。

7. 实施路线(建议)

1. **阶段 0 — 判据先行:** 定义提交追踪接口格式(NDF must 条款),搭 ASL(或其转译)参考模型的锁步对拍框架。在写第一行流水线代码之前,裁判必须先就位。
2. **阶段 1 — 最小可行链:** 选 ISA 极小子集(整数运算 + 分支 + load/store),走通 ASL→NDF 投影→PyCircuit 单发射有序流水线→Verilog→综合,四层测量全部出数,归因链贯通。
3. **阶段 2 — 手动循环:** 人扮演优化智能体跑 5-10 轮完整循环,校准变异目录、度量报表格式、决策记录模板——这些是智能体的"操作手册",先用人验证其可用性。
4. **阶段 3 — 智能体接管内环:** CPI 类假设(旁路、预测器参数)交给智能体自动迭代,人审决策记录;外环(时序重切分)保持人机协同。
5. **阶段 4 — 全循环自动化 + Pareto 档案:** 多分支并行探索,人类角色收缩为规范修订审批与结构性跳变提案。

成功判据与 NDF §11 同构:循环结束时,任何一个 PPA 改进都能回答"哪条条款允许它、哪个决策记录批准它、哪组测量支持它、哪个等价证据担保它"——这时我们得到的不只是一颗更好的核,而是一套可复用、可审计、可持续的核演进机器。

附录 A — 三种表示的等价性速查

	ASL(行为级)	PyCircuit(周期感知)	RTL(可综合)
时间模型	无(指令原子)	显式周期(CycleAwareSignal)	时钟沿
状态空间	架构状态	架构 + 微架构	同 PyCircuit(编译像)
与上一层的等价	—(真理源)	提交点架构轨迹等价(颤动等价)	逐周期等价
主要验证手段	—	锁步对拍 / Burch–Dill / SEC	双后端对拍 / LEC
PPA 可测性	不可	代理可测(结构特征)	直接可测(综合/STA/功耗)
优化动作发生地	否(架构冻结)	是(全部微架构旋钮)	否(仅编译产物)